

CURSO DE ARQUITECTURA DE SOFTWARE

Reto 2: Patrones de Diseño Arquitectónico

Sistema de Farmacia, segunda evolución del diseño TO-BE

Documento de sustentación del equipo
Fecha de entrega: domingo 6 de septiembre de 2026

Índice

ACTIVIDAD 1: Puntos de dolor de la arquitectura actual	2
ACTIVIDAD 2: Decisión de patrones a incorporar	3
Entregable 2.1: tabla de decisión de patrones.....	3
Entregable 2.2: bitácora de decisiones frente a la IA.....	3
ACTIVIDAD 3: Diseño del TO-BE	5
Entregable 3.1: diagrama de dos capas (AS-IS Reto 1 + TO-BE Reto 2 superpuesto).....	5
Entregable 3.2: tabla de cambio estructural	7
Entregable 3.3: fichas de patrón	8
ACTIVIDAD 4: SOLID y comportamiento	11
Entregable 4.1: matriz de verificación SOLID	11
Entregable 4.2: evidencia de que el comportamiento no cambió	11
ACTIVIDAD 5: Análisis de riesgos.....	12
ACTIVIDAD 6: Dos vistas.....	13
Vista para el negocio	13
Vista para el equipo de desarrollo	14
5. Guía de dónde tocar (cubre SC-1, SC-2 y SC-3 del Anexo B).....	15

ACTIVIDAD 1: Puntos de dolor de la arquitectura actual

Reto 2 - Analisis de puntos de dolor

Alcance y criterio

Este analisis toma como AS-IS la solucion de Trabajo Farmacia/03-Src. Solo incluye rigideces relacionadas con creacion, composicion y coordinacion de objetos; no repite defectos funcionales del Reto 1. El costo se mide contando los archivos y clases existentes que hoy deben abrirse o modificarse para atender el escenario descrito.

Cada integrante relevo el codigo por su cuenta antes de consultar cualquier herramienta. P-02, P-03 y P-04 se identificaron asi, sin IA: son puntos que ya se habian notado leyendo ServicioVenta, ServicioMonitoreoProductos y los tres cargadores TXT durante el Reto 1. P-01 y P-05 se consolidaron despues, en la discusion de grupo con apoyo de IA, para confirmar el costo exacto en archivos y clases.

ID	Punto de dolor	Costo actual	Prioridad	Decision
P-01	El formato de productos esta acoplado al orden posicional de sus columnas	3 archivos / 3 clases	Alta	Intervenir
P-02	La venta no admite politicas comerciales variables	4 archivos / 4 clases	Alta	Intervenir
P-03	Cada nueva alerta amplia deteccion, evento y composicion	3 archivos / 3 clases por alerta	Alta	Intervenir
P-04	El algoritmo de carga TXT esta triplicado	3 archivos / 3 clases	Media	Intervenir
P-05	El menu esta centralizado en un switch	1 archivo / 1 clase implicita	Baja	No intervenir

Evidencia detallada (archivo/línea, síntoma y escenario) de cada punto en 1-Puntos-de-Dolor/Puntos_de_Dolor.md, en el repositorio del código entregado.

Conclusion

P-01 encapsula el parseo del inventario en el creador de cada tipo; P-02, P-03 y P-04 justifican respectivamente Strategy, Composite y Template Method como patrones nuevos. P-05 queda como deuda aceptada: demuestra que el equipo no adopta un patron cuando su costo supera el beneficio.

ACTIVIDAD 2: Decisión de patrones a incorporar

Entregable 2.1: tabla de decisión de patrones

Patrones evaluados

Patron	Familia	Punto	Decision	Justificacion
Factory Method	Creacional	P-01	Mantener y consolidar	Sera el unico mecanismo de creacion; ya existe en el AS-IS.
Abstract Factory	Creacional	P-01/P-02	Descartar	No hay familias de objetos que justifiquen varias fabricas relacionadas.
Facade	Estructural	P-02	Descartar	Strategy cubre la variacion sin agregar otra clase coordinadora.
Composite	Estructural	P-03	Adoptar	Permite reunir las alertas sin ampliar el monitor por cada regla nueva.
Strategy	Comportamiento	P-02	Adoptar	Permite cambiar la politica del convenio sin llenar la venta de condicionales.
Template Method	Comportamiento	P-04	Adoptar	Reune en un solo lugar el algoritmo comun de carga TXT.
Chain of Responsibility	Comportamiento	P-03	Descartar	Las alertas deben ejecutarse todas, no detenerse en el primer manejador.
Command	Comportamiento	P-05	Descartar	Agregaria nueve clases para un cambio que hoy se hace en un archivo.

Decision final

Los patrones nuevos seran Strategy, Composite y Template Method. Factory Method se consolidara y Observer seguira como parte del AS-IS, sin contarlos como incorporaciones del Reto 2. Facade y Command quedan descartados porque agregan mas estructura de la que el proyecto necesita.

Entregable 2.2: bitácora de decisiones frente a la IA

ID	Que consultamos	Que propuso la IA	Que hicimos	Argumento del equipo y evidencia
B-01	Cual solicitud de cambio convenia implementar	Trabajar con SC-3 porque introduce distintas reglas comerciales	Aceptamos	SC-3 encaja con P-02. El contrato de IDescuento.cs:9-12 solo recibe un precio y no permite tener en cuenta al cliente, su convenio o el credito.
B-03	Como manejar las distintas politicas de convenio	Aplicar Strategy	Aceptamos	La politica debe elegirse durante la venta sin agregar condicionales por convenio en el flujo principal. Cada regla quedara detras de IPoliticaConvenio.
B-04	Si debiamos crear una estrategia para cada tipo de entidad	Crear estrategias para empresas, bancos, cooperativas, universidades y colegios	Corregimos	La entidad sera un dato. Solo habra estrategias diferentes cuando cambie el calculo: descuento, credito o ambos. Asi evitamos cinco clases con comportamiento casi identico.
B-05	Si Strategy necesitaba una Facade adicional	Crear ServicioConvenios para coordinar las estrategias	Rechazamos	ServicioVentaConvenio puede seleccionar la estrategia directamente. La Facade agregaria dos clases y otra construccion sin ocultar un subsistema que hoy sea complejo.
B-06	Si Facade podia resolver la concentracion de Program	Usarla para reducir el ensamblaje del sistema	Rechazamos	La Facade no elimina ninguno de los objetos ni las suscripciones actuales de Program.cs:8-110; solo moveria u ocultaria ese codigo. Program debe seguir siendo el Composition Root.
B-07	Que patron servia para agregar nuevas alertas	Usar Chain of Responsibility	Corregimos	Las comprobaciones de stock y vencimiento deben ejecutarse siempre. Composite representa mejor un grupo de reglas que se recorren completas, como exige P-03.

B-08	Como evitar la repeticion en los tres cargadores TXT	Aplicar Template Method	Aceptamos	CargadorProductosTxt.cs:22-63, CargadorClientesTxt.cs:13-43 y CargadorUsuariosTxt.cs:13-45 repiten el mismo flujo y solo cambia la conversion de cada linea.
B-09	Si convenia separar cada opcion del menu en un comando	Aplicar Command	Rechazamos	P-05 cuesta hoy un archivo. Command exigiria una interfaz, siete comandos y cambios en Program, sin que existan historial, deshacer u otra interfaz que reutilice esos comandos.
B-10	Si la variacion de convenios justificaba una Abstract Factory	Crear una fabrica abstracta por familia de convenios	Rechazamos	No hay una familia real de objetos que crear de forma coordinada: cada convenio solo cambia el calculo de descuento/credito, no un conjunto de productos relacionados. Una familia irreal agregaria interfaces y fabricas innecesarias, como se registro en el analisis SC-3.
B-11	Si el Observer existente de las alertas debia tratarse como patron nuevo del Reto 2	Contarlo como incorporacion junto a Strategy, Composite y Template Method	Corregimos	Observer ya esta en el AS-IS publicando las alertas de stock y vencimiento, y Composite lo conserva para publicar los mensajes. No es una incorporacion del Reto 2; se mantiene como parte de la base y no se suma a la cuenta de patrones adoptados.
B-12	Si IReglaAlerta debia usar la firma exacta de los metodos actuales del monitor	Una revision externa senalo que Evaluar(producto) invertiria el recorrido y cambiaria el orden de las alertas	Corregimos	Program.cs:182-185 llama VerificarStock(productos) y despues VerificarVencimiento(productos) por lotes completos. Verificar(IEnumerable<Producto>) conserva ese orden exacto; Evaluar(producto) habria intercalado las alertas por producto y roto los 12 casos de caracterizacion.
B-13	Si ServicioMonitoreoProductos debia quedar como coordinador que delega en el compuesto, o salir por completo	Una revision externa senalo que debia salir por completo, no solo transformarse	Aceptamos	Con MonitorCompuesto implementando IReglaAlerta, Program puede invocarlo directamente a traves de ese contrato (DIP). Mantener un servicio que solo reenvia la llamada no agregaria nada, asi que se elimina la clase completa en vez de transformarla.
B-14	Si IDescuento / ServicioDescuento debian marcarse como reemplazados por IPoliticaConvenio	Una revision externa lo afirmo sin evidencia de codigo	Rechazamos primero, luego aceptamos con evidencia propia	CLAUDE.md los declara deuda H-08 que no se debe tratar como reemplazada, asi que se rechazo la afirmacion sin verificarla. Al revisar el codigo real del companero se confirmo que si los elimino. El equipo decidio entonces adoptar esa eliminacion como decision propia, no por la sugerencia externa: IPoliticaConvenio cubre el mismo punto de variacion que ese par dejaba fijo en 10 %, asi que la deuda H-08 queda superada para estas dos clases y CLAUDE.md se actualizo para registrar la excepcion.

ACTIVIDAD 3: Diseño del TO-BE

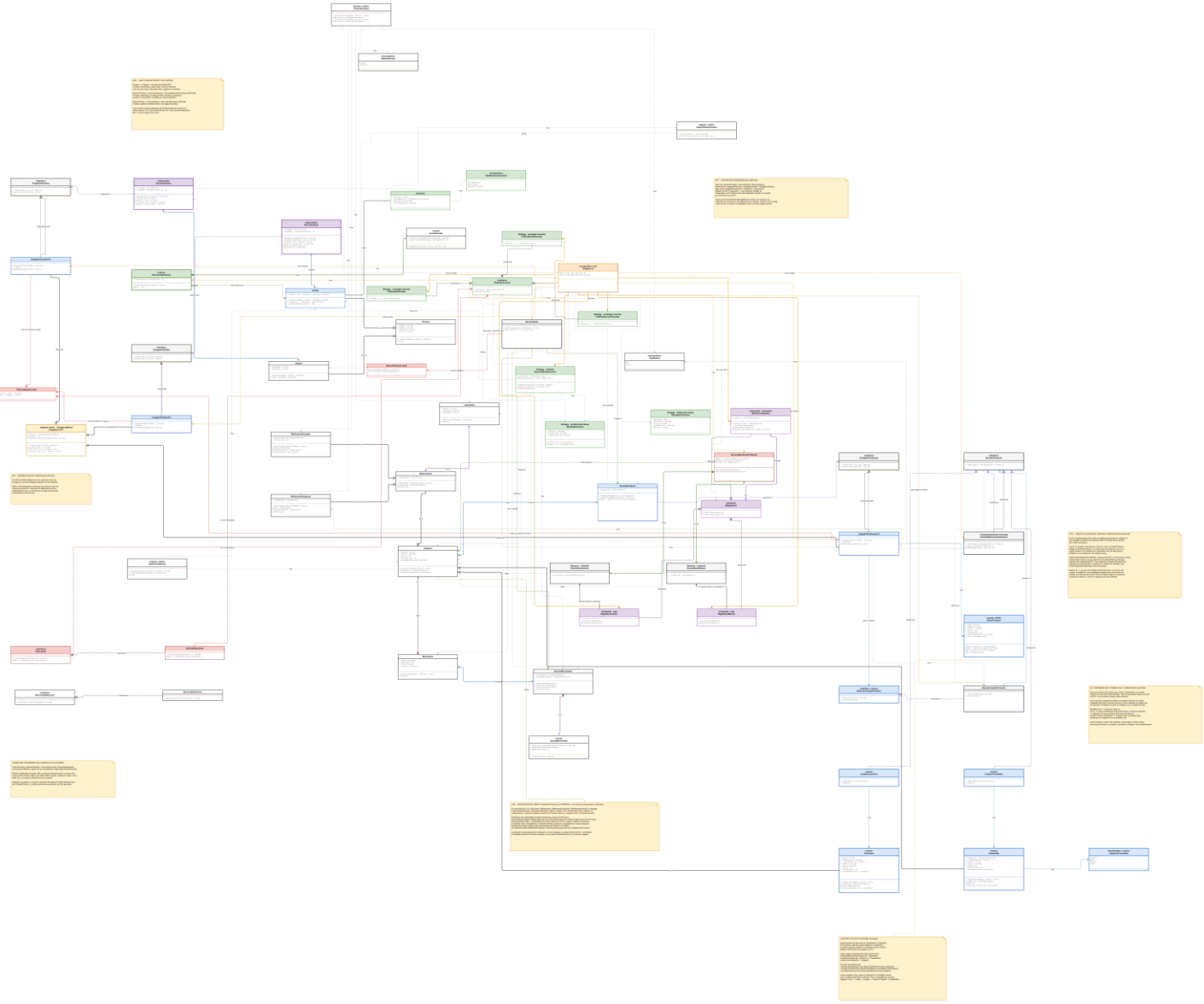
Entregable 3.1: diagrama de dos capas (AS-IS Reto 1 + TO-BE Reto 2 superpuesto)

Capas reales de draw.io: apagando la capa "TO-BE Reto 2" queda solo el Reto 1 sin modificar; encendiéndola, el patrón nuevo aparece exactamente encima de cada clase que toca. Rojo punteado = sale; verde = Strategy; morado = Composite; amarillo = Template Method; gris = se conserva. Fuente vectorial en alta resolución para hacer zoom durante la sustentación: 3-TOBE/1-Diagramas/diagrama_tobe_reto2_overlay.svg y .drawio (editable, capas toggables).

1. The following table lists the components of the system and their interconnections. The components are color-coded according to the legend.

Component	Color
Control Unit	Blue
Power Supply	Red
Signal Processor	Green
Memory Unit	Yellow
Output Device	Purple
Input Device	Orange

Signal	Direction
Control Signal	Control Unit to Signal Processor
Power Signal	Power Supply to Control Unit
Signal Data	Signal Processor to Memory Unit
Memory Data	Memory Unit to Output Device
Input Data	Input Device to Signal Processor



Entregable 3.2: tabla de cambio estructural

ID	Elemento	Estado	Antes -> Ahora	Reconexion
E-01	ServicioVenta.Vender	Transforma	Sin punto de variacion comercial -> mismo flujo exacto; el convenio va por una ruta aparte	Program y los 12 casos siguen igual
E-02	IPoliticaConvenio	Entra	No existia -> contrato Evaluar(SolicitudConvenio): ResultadoConvenio	ServicioVentaConvenio lo consume
E-03	3 politicas concretas	Entran	No existian -> cada una calcula su propio criterio comercial	Registradas en el diccionario de Program
E-04	ServicioVentaConvenio	Entra	No existia -> arma la solicitud, elige politica, aplica el resultado	Program lo construye y expone
E-14	Convenio	Entra	No existia -> Entidad, TipoBeneficio, Porcentaje, CupoDisponible	Cliente lo referencia (0..1)
E-15	TipoBeneficioConvenio	Entra	No existia -> enum SoloDescuento / SoloCredito / DescuentoCredito	Llave del diccionario de politicas
E-16	SolicitudConvenio	Entra	No existia -> solo Subtotal, Porcentaje, CupoDisponible (sin entidades)	ServicioVentaConvenio la arma
E-17	ResultadoConvenio	Entra	No existia -> Aprobado, Descuento, Total, CupoRestante, Motivo	IPoliticaConvenio.Evaluar lo devuelve
E-18	ServicioProducto.BuscarPorNombre	Transforma	No existia aqui -> metodo nuevo, ya duenio de la lista de productos	Program lo usa en 3, 4 y en convenios
E-19	ServicioVenta.BuscarProducto	Sale	Filtraba por nombre -> se traslada intacto a ServicioProducto	Program llama BuscarPorNombre
E-20	Cliente	Transforma	Solo Puntos -> agrega Convenio? por sobrecarga del constructor	Cargadores y servicios sin cambio
E-21	IDescuento	Sale	Contrato de descuento fijo (10%), sin consumidor (H-08) -> superado por IPoliticaConvenio	Nadie lo consumia
E-22	ServicioDescuento	Sale	10% fijo, sin consumidor (H-08) -> variacion la resuelve IPoliticaConvenio	Nadie lo consumia
E-05	ServicioMonitoreoProductos	Sale	2 metodos + 2 eventos propios -> clase completa eliminada	Program usa MonitorCompuesto via IReglaAlerta
E-06	IReglaAlerta	Entra	No existia -> Verificar(IEnumerable<Producto>): void	Hojas y compuesto lo implementan
E-07	ReglaStockMinimo / ReglaVencimiento	Entran	Eran metodos del monitor -> hojas independientes	MonitorCompuesto las compone
E-08	MonitorCompuesto	Entra	No existia -> implementa y compone IReglaAlerta, ejecuta en orden de alta	Program lo arma y lo invoca
E-09	CargadorProductosTxt	Transforma	Flujo completo repetido -> hereda CargadorTxt, solo ParsearCampos	ICargadorProductos y Program sin cambio
E-10	CargadorClientesTxt	Transforma	Idem -> hereda, solo ParsearCampos de Cliente	ICargadorClientes sin cambio
E-11	CargadorUsuariosTxt	Transforma	Idem -> hereda, solo ParsearCampos de Usuario	ICargadorUsuarios sin cambio
E-12	CargadorTxt<T>	Entra	No existia -> algoritmo comun: validar, leer, Split(;), agregar, error	Los 3 cargadores heredan de ella
E-13	ICreadorProducto + SelectorCreadorProducto	Se conserva	Ya era el mecanismo unico de creacion desde Reto 1	Sin cambio; ningun consumidor se rompe

Se eliminan 4 elementos: ServicioVenta.BuscarProducto (E-19, su lógica se traslada intacta), ServicioMonitoreoProductos (E-05, sin rol una vez que Program usa MonitorCompuesto via IReglaAlerta) e IDescuento/ServicioDescuento (E-21, E-22, deuda H-08 superada por IPoliticaConvenio). Ninguna cambia la salida observable. Detalle completo por fila en Tabla_Cambio_Estructural.md.

Entregable 3.3: fichas de patrón

Ficha de patron - Strategy

Patron y punto de dolor que resuelve

Strategy responde a **P-02** (la venta no admite politicas comerciales variables). La evidencia esta en `ServicioVenta.cs:35-52`: Vender descuenta inventario, crea el movimiento y lo registra sin ningun punto donde elegir una regla comercial. `IDescuento.cs:9-12` solo recibe un precio y `ServicioDescuento.cs:11-16` aplica siempre el 10 %. SC-3 exige seleccionar descuento y credito segun empresa, banco, cooperativa o institucion; esa variacion hoy tendria que hundirse en condicionales dentro de `ServicioVenta`.

Alternativas que evaluamos

1. **No hacer nada** (descartada): deja la variacion comercial como condicionales dentro de `ServicioVenta`, el punto rigido que P-02 declara.
2. **Facade** (descartada): `ServicioConvenios` coordinaria las estrategias, pero no hay un subsistema complejo que ocultar; agregaria dos clases sin resolver la seleccion.
3. **Abstract Factory** (descartada): no hay una familia real de objetos que crear de forma coordinada; solo cambia el calculo de la politica.
4. **Strategy** (adoptada): la venta con convenio elige una implementacion de `IPoliticaConvenio` en tiempo de ejecucion.

Que sale y que entra

Sale: la idea de que `ServicioVenta` fije internamente el criterio comercial. Sale tambien `IDescuento / ServicioDescuento` (deuda H-08 en Reto 1, sin consumidor): `IPoliticaConvenio` cubre el mismo punto de variacion con el contexto que a `IDescuento` le faltaba, así que esa deuda queda superada y se elimina en vez de conservarse. Sale `ServicioVenta.BuscarProducto`, que se traslada a `ServicioProducto.BuscarPorNombre`.

Entra: `IPoliticaConvenio (Evaluar(SolicitudConvenio): ResultadoConvenio)` con tres implementaciones: `PoliticaSoloDescuento`, `PoliticaSoloCredito`, `PoliticaDescuentoCredito`. Entra `ServicioVentaConvenio`, que selecciona la politica y aplica su resultado. Entran `Convenio (Entidad, TipoBeneficio, Porcentaje, CupoDisponible)`, `TipoBeneficioConvenio (enumeracion)`, `SolicitudConvenio (Subtotal, Porcentaje, CupoDisponible)` y `ResultadoConvenio (Aprobado, Descuento, Total, CupoRestante, Motivo)`.

La solicitud y el resultado llevan solo numeros y banderas, nunca `Cliente`, `Producto` ni `Convenio`: si los cargaran, una politica podria mutar el dominio. Quien descuenta stock, registra el movimiento y actualiza el cupo es siempre `ServicioVentaConvenio`, solo cuando `Aprobado = true`. `Aprobado` es general (no `CreditoAutorizado`) porque `PoliticaSoloDescuento` no evalua credito.

Como se relaciona

Program registra en un diccionario `IDictionary<TipoBeneficioConvenio, IPoliticaConvenio>` las tres politicas y lo pasa a `ServicioVentaConvenio`. `Cliente` gana `Convenio: Convenio? (0..1)` por una sobrecarga del constructor, sin romper a los consumidores actuales. `VenderConConvenio(cliente, producto, cantidad)` arma la `SolicitudConvenio`, elige la politica por `TipoBeneficio`, la invoca y, si aprueba, descuenta stock y registra el movimiento en `ServicioMovimiento`. El producto se ubica con `ServicioProducto.BuscarPorNombre`. La venta normal no participa. Los rechazos (cliente sin convenio, cantidad invalida, cupo insuficiente) devuelven `Aprobado = false` con su `Motivo`, sin excepciones: verificado en `Evidencia_4_2.md` (casos CN-01 a CN-04).

Impacto

Creadas: `IPoliticaConvenio` y sus 3 implementaciones, `ServicioVentaConvenio`, `Convenio`, `TipoBeneficioConvenio`, `SolicitudConvenio`, `ResultadoConvenio`. Modificadas: `Program`, `Cliente` (agrega `Convenio` opcional), `ServicioProducto` (agrega `BuscarPorNombre`). Eliminadas: `IDescuento`, `ServicioDescuento`, y el metodo `ServicioVenta.BuscarProducto`. Habilita SC-3 sin tocar la venta normal; SC-1 y SC-2 no se ven afectados.

Que cuesta

Mas indireccion: una interfaz, tres estrategias y un servicio nuevo. La venta con convenio agrega una capa que la venta simple no tiene. Es el costo de un punto de variacion real, que solo se justifica porque SC-3 lo exige.

Origen

Propuesta de la herramienta aceptada tras verificarla contra el codigo. Bitacora **B-03** (Strategy para convenios), **B-04** (una estrategia por tipo de calculo, no por entidad) y **B-05** (rechazo de la Facade adicional).

Ficha de patron - Composite

Patron y punto de dolor que resuelve

Composite responde a **P-03** (cada nueva alerta amplia deteccion, evento y composicion). La evidencia esta en `ServicioMonitoreoProductos.cs:20-31` (`VerificarStock`) y `33-48` (`VerificarVencimiento`): el monitor recorre los productos por separado para cada regla y dispara un evento concreto por regla. Agregar una alerta de sobrestock obliga a crear otro evento, ampliar el monitor y rehacer la composicion en `Program`.

Alternativas que evaluamos

1. **No hacer nada** (descartada): cada regla de alerta nueva seguiria tocando deteccion, evento y ensamblaje a la vez, que es el costo repetido que P-03 declara.

2. **Chain of Responsibility** (descartada): una cadena detiene la revisión en el primer manejador que la atiende, pero las alertas de stock y vencimiento deben ejecutarse siempre. No representa bien un grupo de reglas que se recorren completas.
3. **Composite** (adoptada): agrupa las reglas bajo un contrato común y las ejecuta todas, sin que el monitor crezca con cada regla nueva.

Que sale y que entra

Sale: la clase completa `ServicioMonitoreoProductos`. No queda ni como coordinador ni como delegador intermedio: una vez que `MonitorCompuesto` implementa `IReglaAlerta`, `Program` puede invocarlo directamente a través de ese contrato (DIP), y mantener un servicio que solo reenvía la llamada no aportaría nada.

Entra: `IReglaAlerta` (contrato común `Verificar(productos: IEnumerable<Producto>): void`), las hojas `ReglaStockMinimo` y `ReglaVencimiento` (cada una recibe la colección completa, la recorre y publica sus mensajes), y `MonitorCompuesto` (implementa `IReglaAlerta` además de componer `IReglaAlerta`: al verificar, ejecuta cada regla de su lista completa y en orden de alta). El mecanismo `Observer` existente que publica los mensajes se conserva igual, tal como ya está en el AS-IS.

La operación recibe la colección, no un producto suelto, y esa decisión es de conservación de conducta: hoy `Program.cs:182-185` llama `VerificarStock(productos)` y luego `VerificarVencimiento(productos)`, de modo que la salida sale agrupada por regla (todas las alertas de stock, después todas las de vencimiento). Un contrato `Evaluar(producto)` invertiría el recorrido y los mensajes quedarían intercalados por producto, cambiando la salida observable y rompiendo los 12 casos de caracterización. Con `Verificar(IEnumerable<Producto>)` la firma es idéntica a la de los métodos actuales (`ServicioMonitoreoProductos.cs:20 y 33`) y el orden se conserva exactamente.

Como se relaciona

`Program` arma el `MonitorCompuesto`, le agrega las reglas, y lo invoca directamente como `IReglaAlerta.Verificar(productos)` (antes llamaba a `ServicioMonitoreoProductos`, que ya no existe); sigue suscribiéndose a la notificación de cada alerta. El compuesto reenvía la colección completa a cada regla de su lista. Una alerta nueva es una clase hoja que implementa `IReglaAlerta` y un registro en `Program`; no se toca el algoritmo de recorrido ni la composición de `Program` más allá del alta. Se apoya en el `Observer` ya existente para publicar, sin introducir un canal nuevo.

Impacto

Clases creadas: `IReglaAlerta`, `ReglaStockMinimo`, `ReglaVencimiento`, `MonitorCompuesto`. Clases modificadas: `Program` (deja de construir `ServicioMonitoreoProductos`, arma y usa `MonitorCompuesto` como `IReglaAlerta`). Clases eliminadas: `ServicioMonitoreoProductos` completa; los eventos y el `Observer` se conservan sin cambios. Efecto sobre las solicitudes del Anexo B: no altera la conducta de las alertas existentes ni su orden; una alerta nueva (p. ej. de sobrestock) se agrega como hoja.

Que cuesta

Se paga una clase base (`IReglaAlerta`), dos reglas, el monitor compuesto y el reensamblaje de `Program`. La composición por reglas agrega indirectamente: leer donde se decide el umbral ya no es un método del monitor sino la regla. Es el costo de dejar de ampliar estructuras cada vez que aparece una regla de alerta.

Origen

Propuesta de la herramienta corregida y adoptada tras verificarla. Queda registrada en la bitácora como **B-07** (la herramienta sugirió `Chain of Responsibility`; el equipo la reemplazó por `Composite` porque las alertas deben ejecutarse todas).

Ficha de patron - Template Method

Patron y punto de dolor que resuelve

`Template Method` responde a **P-04** (el algoritmo de carga TXT está triplicado). La evidencia está en la misma estructura repetida de `CargadorProductosTxt.cs:22-63`, `CargadorClientesTxt.cs:13-43` y `CargadorUsuariosTxt.cs:13-45`: los tres verifican `File.Exists`, leen las líneas, recorren, aplican `Split(';')`, construyen una entidad, la agregan al destino y capturan la excepción. Solo cambia la conversión de una fila a su entidad.

Alternativas que evaluamos

1. **No hacer nada** (descartada): una regla común de formato (aceptar encabezados, ignorar comentarios) seguiría teniendo que implementarse y mantenerse sincronizada en tres archivos separados, con riesgo de divergir.
2. **Una utilidad estática compartida** (descartada): concentraría la lectura en un helper, pero obligaría a cada cargador a recordar llamar a cada paso; un consumidor que salte un paso escala mal y reparte el orden lógico del algoritmo fuera del flujo.
3. **Template Method** (adoptada): una clase base `CargadorTxt<T>` fija el flujo completo en un solo lugar y deja la conversión de los campos ya separados como paso especializado (`ParsearCampos`).

Que sale y que entra

Sale: la duplicación del flujo de lectura y manejo de errores en los tres cargadores.

Entra: `CargadorTxt<T>` (clase base que contiene el algoritmo común: validar, leer, recorrer, aplicar `Split(';')`, agregar, manejar error) con dos puntos de variación: el paso abstracto `ParsearCampos(string[] campos): T` que cada cargador concreto implementa, y el hook `protected abstract string MensajeCargaExitosa { get; }`. `CargadorProductosTxt`, `CargadorClientesTxt` y `CargadorUsuariosTxt` pasan a heredar de la base y solo conservan su conversión de campos y su mensaje.

El separador queda en la base y no en las subclases: el `Split(';')` es idéntico en los tres cargadores, así que forma parte del algoritmo común. Si el paso abstracto recibiera la línea cruda, cada subclase tendría que volver a partirla y el separador seguiría repetido tres veces, que es justo el costo que **P-04** declara. El hook `MensajeCargaExitosa` existe porque cada cargador devuelve hoy un texto distinto al terminar ("`Productos cargados`", "`Clientes cargados`", "`Usuarios cargados`") y ese texto es salida observable que la caracterización compara.

Como se relaciona

Program sigue construyendo los cargadores concretos tal como hoy, porque cada uno conserva su interfaz (ICargadorProductos, ICargadorClientes, ICargadorUsuarios). Al llamar Cargar, la clase base ejecuta el flujo comun y delega en ParsearCampos. No interactua con otro patron adoptado; es una reorganizacion interna de la carga. Se cuida que el paso de la plantilla sea comun para todos y no se agreguen pasos vacios que una subclase no pueda cumplir.

Impacto

Clases creadas: CargadorTxt<T>. Clases modificadas: CargadorProductosTxt, CargadorClientesTxt, CargadorUsuariosTxt (pasan a heredar y recortan su metodo al parseo). Clases eliminadas: ninguna interfaz: las tres interfaces ICargador* se conservan para no alterar a sus consumidores. Efecto sobre las solicitudes del Anexo B: un cambio transversal de formato se hace una vez en la base; SC-1 (nuevos tipos de producto) se simplifica porque el parseo de fila queda aislado en cada creador o en el cargador concreto.

Que cuesta

Se paga una clase base nueva y la transformacion de los tres cargadores. El flujo ahora vive en la herencia, lo que exige disciplina: si una subclase necesita un paso distinto, modificar la plantilla afecta a los tres, por lo que los pasos deben seguir siendo comunes. Es el costo de eliminar la triple copia.

Origen

Propuesta de la herramienta aceptada tras verificarla contra el codigo. Queda registrada en la bitacora como **B-08** (Template Method para la repeticion de los tres cargadores TXT).

ACTIVIDAD 4: SOLID y comportamiento

Entregable 4.1: matriz de verificación SOLID

Patrón adoptado	SRP	OCP	LSP	ISP	DIP
Strategy (IPoliticaConvenio)	Refuerza	Refuerza	Neutro	Tensionado, compensado	Refuerza
Composite (IReglaAlerta)	Refuerza	Refuerza	Tensionado, compensado	Neutro	Refuerza
Template Method (CargadorTxt<T>)	Neutro	Refuerza	Tensionado, compensado	Neutro	Neutro
Factory Method (consolidacion)	Neutro	Refuerza	Neutro	Neutro	Refuerza

Ninguna celda queda en Roto. Línea de evidencia bajo cada celda distinta de Neutro:

- Strategy / SRP (Refuerza): cada política calcula un solo criterio comercial; antes esa variación caía dentro de ServicioVenta.Vender.
- Strategy / OCP (Refuerza): una política nueva es una clase + un registro en Program; no se toca ServicioVenta ni la salida de los 12 casos.
- Strategy / ISP (Tensionado, compensado): Evaluar() devuelve 6 datos; se acota a evaluación comercial pura, sin persistencia ni interfaz (Ficha_Strategy).
- Strategy / DIP (Refuerza): ServicioVentaConvenio depende de IPoliticaConvenio, no de una política concreta; Program decide cual inyectar.
- Composite / SRP (Refuerza): cada regla (stock, vencimiento) encapsula su propia detección; antes eran métodos del mismo monitor.
- Composite / OCP (Refuerza): una alerta nueva es una hoja + su alta en Program; no se amplía el monitor existente.
- Composite / LSP (Tensionado, compensado): todas las reglas se ejecutan igual bajo MonitorCompuesto; se evita que una hoja necesite un paso que otras no cumplan.
- Composite / DIP (Refuerza): MonitorCompuesto depende de IReglaAlerta para componer; Program no conoce las hojas concretas.
- Template Method / OCP (Refuerza): un cargador nuevo o una regla de formato se atiende en CargadorTxt<T>, no en tres archivos.
- Template Method / LSP (Tensionado, compensado): riesgo de un paso vacío en una subclase; los pasos de la plantilla son comunes y obligatorios, no opcionales.
- Factory Method / OCP (Refuerza): un tipo de producto nuevo es un creador + un registro en el diccionario; CargadorProductosTxt no se modifica.
- Factory Method / DIP (Refuerza): CargadorProductosTxt depende de ISelectorCreadorProducto / ICreadorProducto, no de creadores concretos.

Errores típicos verificados

Situación que evita el enunciado	Principio	Estado en el TO-BE
Fabrica que crece con un condicional por tipo nuevo	OCP	No ocurre: SelectorCreadorProducto usa un diccionario, no condicionales
Fachada que absorbe lógica de negocio	SRP	No se adopta Facade (descartada, B-05/B-06)
Punto de acceso global de instancia única	DIP	No se usa Singleton
Metodo plantilla con un paso que una subclase no cumple	LSP	Declarado y compensado en Template Method
Envoltura que cambia el contrato de lo que envuelve	LSP	Las interfaces ICargador* se conservan sin cambios

Entregable 4.2: evidencia de que el comportamiento no cambió

12 casos heredados del Reto 1, corridos contra el TO-BE Reto 1 (03-src) y contra el código real del Reto 2, comparados byte a byte, más 4 casos nuevos que ejercitan Strategy, el único patrón que agrega comportamiento observable (Composite, Template Method y Factory Method se adoptaron para que la salida no cambiara, y la prueba de eso son los 12 casos heredados, no un camino nuevo).

Caso	Escenario	Resultado
CC-01	Arranque: carga de los tres archivos y alertas iniciales	Idéntica
CC-02	Login con credenciales válidas	Idéntica
CC-03	Login con credenciales inválidas	Idéntica
CC-04	Listar productos	Idéntica
CC-05	Listar clientes con sus puntos	Idéntica
CC-06	Buscar un producto que existe	Idéntica
CC-07	Buscar un producto que no existe	Idéntica
CC-08	Venta con stock suficiente	Idéntica
CC-09	Venta mayor al stock disponible (H-07 se reproduce igual)	Idéntica
CC-10	Acumular puntos a un cliente	Idéntica
CC-11	Ver alertas de stock mínimo y vencimiento (pasa por MonitorCompuesto)	Idéntica
CC-12	Opción de menú que no existe	Idéntica

Resultado: 12 de 12 casos con salida idéntica. CC-01 y CC-11 prueban Template Method y Composite (misma salida con la carga y las alertas reorganizadas); CC-04 prueba Factory Method (inventario completo sin cambios).

Caso	Escenario	Convenio	Cálculo esperado	Resultado real
CN-01	PoliticaSoloDescuento aprobada	Carlos, EmpresaAcme, 10 %, cupo 0	Omeprazol x2 = 30000; descuento 10 % = 3000; total 27000; cupo restante 0 (no se	Subtotal 30000 / Descuento 3000 / Total 27000 / Cupo restante 0 (coincide)

			toca en esta política)	
CN-02	PoliticaSoloCredito aprobada	Ana, BancoCentral, cupo 20000	Dolex x2 = 10000; sin descuento; cupo alcanza: cupo restante 20000 – 10000 = 10000	Subtotal 10000 / Descuento 0 / Total 10000 / Cupo restante 10000 (coincide)
CN-03	PoliticaDescuentoCredito aprobada	Juan, UniversidadCentral, 10 %, cupo 20000	Acetaminofen x3 = 13500; descuento 10 % = 1350; total 12150; cupo alcanza: cupo restante 20000 – 12150 = 7850	Subtotal 13500 / Descuento 1350 / Total 12150 / Cupo restante 7850 (coincide)
CN-04	Rechazo: cliente sin convenio	Maria (cédula 741), no tiene convenio	ServicioVentaConvenio debe rechazar antes de tocar cualquier política, con Motivo = "Cliente sin convenio"	"Venta con convenio rechazada: Cliente sin convenio" (coincide)

En CN-01 a CN-03 se dispara el evento MovimientoRegistrado (Observer existente): la venta con convenio sí descuenta stock y registra el movimiento cuando aprueba. En CN-04 no se dispara: el rechazo corta el flujo antes de tocar inventario. Salidas completas en evidencia-4.2/.

ACTIVIDAD 5: Análisis de riesgos

Riesgos de incorporar los patrones al código del Reto 1, como condición y consecuencia, con probabilidad, impacto, exposición (P×I), mitigación y señal observable.

ID	Riesgo (si X, entonces Y)	P	I	Exp	Mitigacion	Senial observable
R-01	CargadorTxt<T> introduce un paso que un cargador no puede cumplir -> LSP roto, una carga falla o cambia	3	4	12	Pasos comunes y obligatorios; cada cargador solo implementa ParsearCampos	Falta un mensaje de carga o CC-01 deja de coincidir
R-02	IPoliticaConvenio crece con demasiados datos en Evaluar -> ISP tensionado	2	3	6	Contrato limitado a evaluacion comercial, sin persistencia ni interfaz	Una política necesita datos fuera del contrato
R-03	Composite altera orden o contenido de las alertas -> sale observable cambia, fallan los 12 casos	2	5	10	Mismo criterio y mensaje; Observer sin cambios; 12 casos comparados por hash	Una alerta cambia de orden, color o desaparece (CC-11)
R-04	Error en Program rompe el ensamblaje -> el sistema no arranca o un servicio queda mal conectado	2	4	8	Program es el unico ensamblador; se compila y prueba tras cada cambio	Excepcion al arrancar o una opcion de menu no responde
R-05	Se adopta un patron sin un punto rigido que lo justifique -> sobre-ingenieria, - 0.3 por patron	2	3	6	Cada patron se ancla a un P-xx y a una solicitud del Anexo B	Auditoria encuentra un patron o clase sin justificacion
R-06	Factory Method se consolida sin resolver el parseo posicional -> P-01 sigue abierto	4	2	8	Se declara como deuda consciente, no silenciosa	Agregar una columna sigue costando 3 archivos / 3 clases

Mayor exposición: **R-01** (12, LSP en Template Method) y **R-03** (10, salida de alertas); ambos se moderan corriendo y comparando los 12 casos por hash. Ningún riesgo exige cambiar estilo arquitectónico, framework, persistencia o interfaz gráfica.

ACTIVIDAD 6: Dos vistas

Vista para el negocio

Para: la Dirección de Ingeniería y quien aprueba el presupuesto.

Qué le vamos a hacer al sistema y qué no cambia

No tocamos nada de lo que el sistema ya hace hoy: cada venta, cada cálculo y cada alerta de inventario siguen funcionando exactamente igual que antes.

Lo comprobamos ejecutando los mismos doce escenarios de prueba que se usaron en la entrega anterior y comparando la salida línea por línea: coincide al cien por ciento.

Lo único nuevo es la capacidad de manejar los convenios comerciales que ustedes pidieron: descuentos y crédito con empresas, bancos, cooperativas, universidades y colegios. Es una opción adicional, no reemplaza la venta normal.

Dónde se está yendo hoy el tiempo y el dinero

Cada vez que aparece un tipo de alerta de inventario nuevo, hoy hay que tocar varios archivos a la vez para agregarla, y nadie puede simplemente sumar una regla nueva sin volver a armar esa parte completa.

La forma en que hoy se cargan los archivos de productos, clientes y usuarios repite el mismo procedimiento tres veces por separado. Si mañana cambia el formato de esos archivos, hay que corregirlo tres veces en vez de una, con el riesgo de que alguna copia quede desactualizada.

La venta tampoco tenía ningún lugar preparado para aplicar una regla comercial distinta según el cliente. Si ustedes hubieran pedido el manejo de convenios sobre el diseño anterior, esa lógica habría tenido que meterse a la fuerza dentro del proceso central de venta, haciéndolo más frágil con cada convenio nuevo.

Qué gana el negocio

Agregar una alerta de inventario nueva deja de obligar a tocar varios archivos: se suma una pieza nueva sin rehacer las que ya funcionan. Eso baja el tiempo de respuesta la próxima vez que pidan una alerta adicional.

Agregar un convenio nuevo con otra entidad no obliga a tocar el proceso de venta: se agrega por separado y se conecta en un solo punto. El riesgo de que un convenio nuevo rompa una venta normal queda controlado.

En general, el riesgo de que un cambio futuro rompa algo que ya funciona baja, porque cada parte del sistema queda más aislada de las demás.

Qué cuesta

Este cambio no agrega tiempo de desarrollo visible del lado del cliente ni de la operación diaria: es trabajo interno de organización, no una funcionalidad aparte además del manejo de convenios.

Sí cuesta algo de complejidad interna: hay más piezas pequeñas que antes (donde había una sola pieza encargada de las alertas de inventario, ahora hay varias piezas más chicas que se combinan). Eso se paga una sola vez ahora, para no pagarlo cada vez que aparezca una alerta o un convenio nuevo.

Qué riesgos hay

El riesgo principal es que, al reorganizar cómo se arman las alertas de inventario, cambie por accidente el orden o el contenido de un mensaje que el negocio ya conoce. Lo mitigamos comparando la salida del sistema, línea por línea, contra la versión anterior, antes de entregar.

Otro riesgo es que el sistema no arranque si algo queda mal conectado internamente. Lo mitigamos probando después de cada cambio, no solo al final.

Ambos riesgos se muestran resueltos en la sustentación, con el sistema corriendo en vivo.

Qué necesitamos del negocio

Confirmar los datos reales de los convenios que se van a manejar (con qué entidades, qué porcentaje de descuento, qué cupo de crédito), porque hoy se trabaja con datos de ejemplo mientras se valida el diseño.

Aprobar que el alcance de esta entrega se quede en la lógica interna del sistema: no se está tocando base de datos, ni la interfaz, ni la conectividad con otros sistemas.

Qué pasa si no se hace

El sistema sigue funcionando para lo que ya hace, pero cada solicitud nueva (una alerta, un convenio, un producto de otro tipo) va a seguir costando más tiempo del necesario, porque cada una obliga a tocar partes que ya funcionan en vez de sumar una pieza aparte. El costo no desaparece: se sigue acumulando en cada entrega futura.

Vista para el equipo de desarrollo

Para: el ingeniero que entra al equipo dentro de seis meses y tiene que hacer un cambio sin romper nada. No estuvo en ninguna reunión de diseño.

1. Qué patrones hay, dónde vive cada uno y cómo se relacionan

Strategy (SC-3, convenios) vive en `BibFarmacia/Servicios/ Interfaces/`. `IPoliticaConvenio` es el contrato (``Evaluar(SolicitudConvenio): ResultadoConvenio``); `PoliticaSoloDescuento`, `PoliticaSoloCredito`` y `PoliticaDescuentoCredito` son las tres implementaciones. `ServicioVentaConvenio` es el contexto: recibe un diccionario ``IDictionary<TipoBeneficioConvenio, IPoliticaConvenio>`, elige la política según el `TipoBeneficio`` del convenio del cliente y aplica su resultado. Los tipos de datos (`Convenio`, `TipoBeneficioConvenio`, `SolicitudConvenio`, `ResultadoConvenio`) están en `BibFarmacia/Clases/` y `Enums/`.

Composite (alertas de stock y vencimiento) vive en `BibFarmacia/Servicios/`. `IReglaAlerta` es el contrato común (`Verificar(IEnumerable<Producto>): void`); `ReglaStockMinimo` y `ReglaVencimiento` son las hojas; `MonitorCompuesto` agrupa una lista de `IReglaAlerta` y también implementa `IReglaAlerta`, por eso `Program` lo consume igual que consumiría una regla suelta sin saber que por dentro reenvía a varias.

Template Method (carga de TXT) vive en `BibFarmacia/Servicios/`. `CargadorTxt<T>` es la clase base con el algoritmo fijo (validar archivo, leer líneas, recorrer, separar por `;`, agregar al destino, capturar error); `CargadorProductosTxt`, `CargadorClientesTxt` y `CargadorUsuariosTxt` heredan de ella y solo implementan `ParsearCampos(string[])`: `T` y el hook `MensajeCargaExitosa`.

Factory Method (consolidado, ya existía en Reto 1) vive en `BibFarmacia/Factories/`. `ICreadorProducto` es el contrato, `CreadorMedicamentoCapsula` / `CreadorCosmetico` / `CreadorComestible` las implementaciones, `SelectorCreadorProducto` elige cuál usar según el tipo leído de productos. `txt`. No cambió en este reto.

2. Dónde se ensambla el sistema

`AppFarmaciaConsola/Program.cs` sigue siendo el único `Composition Root`: es el único archivo que hace `new` de cada servicio concreto, arma el diccionario de políticas de convenio, arma el `MonitorCompuesto` con sus dos

reglas y conecta todos los eventos. Ningún servicio de BibFarmacia construye a otro directamente. Si algo no arranca, el primer lugar donde mirar es Program.cs.

Program depende de MonitorCompuesto solo a través de IReglaAlerta (IReglaAlerta monitorAlertas = monitorCompuesto;); no conoce las reglas concretas ni cómo están compuestas.

3. Reglas que no se deben romper y por qué

- **La venta normal (`ServicioVenta.Vender``) no se toca.** Es el flujo que validan los 12 casos de caracterización (ver Evidencia_4_2.md). La venta con convenio es una ruta aparte (`ServicioVentaConvenio`) que no pasa por `ServicioVenta`.
- **SolicitudConvenio y ResultadoConvenio solo llevan números y banderas, nunca Cliente, Producto ni Convenio.**** Si una política recibiera esas entidades podría mutarlas y dejaría de ser una estrategia de cálculo puro. Quien descuenta stock, registra el movimiento y actualiza el cupo es siempre `ServicioVentaConvenio`, nunca la política.
- **El separador `;` y el flujo de lectura viven solo en `CargadorTxt<T>`.** Si una subclase vuelve a hacer `Split(';')` por su cuenta, se reintroduce la triplicación que Template Method vino a resolver.
- **ProductoFactory, ServicioNotificacion + IServicioNotificacion y AspectoValidacion no se conectan ni se borran.**** Es deuda declarada del Reto 1 (sin consumidor, ver H-08); no forman parte del diseño vigente pero se conservan para la comparación AS-IS/TO-BE.

4. Deuda pendiente

- **`IDescuento`` y `ServicioDescuento`` se eliminaron** en este reto: cubrían el mismo punto de variación comercial que ahora resuelve `IPoliticaConvenio`, así que dejaron de tener sentido como deuda separada.
- **El formato del archivo de productos sigue acoplado por posición** (`datos[0]` es el tipo, `datos[1]` el nombre, etc.). Consolidar `Factory Method` no resolvió eso: cada creador sigue leyendo `Extra[i]` por índice. Agregar una columna nueva al inventario sigue costando tocar `DatosProducto`, `CargadorProductosTxt` y cada creador (P-01 sigue abierto).
- **El menú de `Program`` sigue siendo un único `switch`` (P-05):** se evaluó y se descartó a propósito resolverlo con un patrón, porque el costo de la alternativa (nueve clases) superaba el problema (un archivo).

5. Guía de dónde tocar (cubre SC-1, SC-2 y SC-3 del Anexo B)

Si el cambio es...	Qué crear	Qué modificar	Qué NO tocar
Un tipo de producto nuevo (SC-1: cosméticos, comestibles)	Una clase de dominio que herede de <code>Producto</code> + una implementación de <code>ICreadorProducto</code>	Un registro nuevo en el diccionario de <code>SelectorCreadorProducto</code> , en <code>Program</code>	<code>CargadorProductosTxt</code> , <code>DatosProducto</code> , los demás creadores
Un servicio vendible nuevo (SC-2: inyectología, curaciones)	Depende de si se modela como <code>Producto</code> (reutiliza la venta) o como un flujo aparte, como se hizo con convenios	El punto de ensamblaje en <code>Program</code> para exponer la opción de menú	<code>ServicioVenta.Vender</code> , la venta normal existente
Un convenio con una entidad nueva (SC-3: otro banco, otra cooperativa)	Nada de código: es un dato, una instancia de <code>Convenio</code> con su <code>TipoBeneficio</code> y <code>Porcentaje</code>	La fuente de datos de clientes (<code>clientes.txt</code>), si el convenio se carga desde ahí	<code>IPoliticaConvenio</code> , sus tres implementaciones, <code>ServicioVentaConvenio</code>
Una regla de cálculo comercial nueva (ej. combinar tres beneficios)	Una clase que implemente <code>IPoliticaConvenio</code>	Su registro en el diccionario de políticas, en <code>Program</code>	<code>ServicioVentaConvenio</code> , las demás políticas
Una alerta de inventario nueva (ej. sobrestock)	Una clase que implemente <code>IReglaAlerta</code>	Su alta (<code>Agregar(...)</code>) en <code>MonitorCompuesto</code> , en <code>Program</code>	<code>IReglaAlerta</code> , <code>ReglaStockMinimo</code> , <code>ReglaVencimiento</code> , el algoritmo de recorrido
Una regla de formato nueva para los TXT (ej. ignorar líneas que empiecen con #)	Nada nuevo	El algoritmo común en <code>CargadorTxt<T>.Cargar</code>	Los tres <code>ParsearCampos</code> de las subclases: no deberían saber de esto